

CanlıFal — Faz 15 Raporu

WebRTC Telemetri (§76) + Gözlemlenebilirlik (§74)

Tarih: 2026-08-27

Checkpoint: Phase 15: WebRTC telemetry observability

Doğrulama: tsc  / build  / dev sunucu  / canlı uç doğrulaması  (401 + 405 zarfları) / tarayıcı UI testi  / deploy 

1. Amaç

Master mimarinin §76 (WebRTC kalite/telemetri) ve §74 (gözlemlenebilirlik) maddelerinde belirtilen, backend'de hiç bulunmayan bağlantı kalite telemetrisi katmanını eklemek.

Kısıt: Tamamen eklemeli. Mevcut hiçbir uç noktanın davranışı değiştirilmedi, medya taşıma mimarisine ve sinyalleşme akışına dokunulmadı.

2. Yapılan değişiklikler

Dosya	İşlem
prisma/schema.prisma	RtcTelemetry modeli eklendi (ilişkisiz, 5 indeks, @@map("rtc_telemetry"))
lib/rtc-telemetry.ts	YENİ — skorum + kayıt + sorgulama katmanı
lib/rate-limit-guard.ts	rtc_telemetry: 30 scope'u eklendi
app/api/rtc/telemetry/route.ts	YENİ — POST ingest ucu
app/api/admin/rtc-telemetry/route.ts	YENİ — GET admin liste + özet
app/[lang]/admin/rtc-telemetry/page.tsx	YENİ — admin arayüzü
app/[lang]/admin/page.tsx	Navigasyona  Bağlantı Kalitesi bağlantısı
scripts/seed.ts	rtc_quality_thresholds RemoteConfig kaydı
docs/CANLIFAL_WEBRTC.md	§10 bölümü "Planlanan" → "Uygulandı" olarak yeniden yazıldı

3. RtcTelemetry modeli

Yalnızca ekleme yapılan (append-only), ilişkisiz bir tablodur; mevcut modellerle yabancı anahtar bağı kurulmaz, dolayısıyla geriye dönük uyumluluk riski yoktur.

İndeksler: `userId+createdAt` , `context+createdAt` , `contextId` , `qualityLevel+createdAt` , `createdAt` .

Saklanan alanlar: `context` , `contextId` , `peerId` , `connectionState` , `iceState` , `reconnectCount` , `rttMs` , `packetLossPercent` , `jitterMs` , `bitrateKbps` , `freezeCount` , `freezeDurationMs` , `durationSeconds` , `platform` , `networkType` , `qualityScore` , `qualityLevel` , `metadata` .

Gizlilik: IP adresi, çerez veya ses/görüntü içeriği saklanmaz.

4. Kalite skorlaması

`computeRtcQuality(input)` 0-100 arası ağırlıklı skor üretir:

Sinyal	Ağırlık	iyi	orta	kötü
RTT (ms)	30	≤150	≤300	≤500
Paket kaybı (%)	30	≤1	≤3	≤8
Jitter (ms)	20	≤30	≤60	≤100
Yeniden bağlanma	10	0	≤1	≤3
Donma sayısı	10	0	≤2	≤5

Seviyeler: `excellent` ≥85, `good` ≥70, `fair` ≥50, `poor` ≥30, altı `critical` .

Eşikler `rtc_quality_thresholds` RemoteConfig kaydı ile ezilebilir (60 sn önbellek).
Kayıt okunamazsa sessizce varsayılanlara düşer, asla hata fırlatmaz.

Eksik metrikler skorlamadan çıkarılır ve ağırlıklar kalan sinyaller üzerinden yeniden normalize edilir — böylece kısmi veri gönderen istemciler cezalandırılmaz.

Değer kırpması: rtt 0-60000, kayıp 0-100, jitter 0-10000, donma 0-100000 ms.
Kötü niyetli istemcinin uç değerlerle özet istatistikleri bozmasını engeller.

5. Uç noktalar

POST /api/rtc/telemetry (ve /api/v1/rtc/telemetry)

- Auth: oturum gerekli
- Rate limit: `rtc_telemetry` scope, 30/dk

- Tekil gövde veya `{ samples: [...] }` ile toplu gönderim (en fazla 20 örüntü)
- Yanıt: `apiSuccess` zarfı; geçersiz gövde `VALIDATION_ERROR`, oturumsuz `UNAUTHORIZED`

GET /api/admin/rtc-telemetry

- Auth: admin rolü (`resolveUser` + `isAdminRole`)
- Filtreler: `context`, `level`, `userId`, `hours`, `page`, `limit`
- Yanıt: sayfalı liste + özet (ortalama skor/RTT/kayıp/jitter, seviye dağılımı)

6. Yan bulgu — ölü /api/v1/* rotaları düzeltildi

Canlı doğrulama sırasında `/api/v1/*` altındaki **tüm** uçların 404 döndürdüğü tespit edildi. Nedeni: `middleware.ts` `/api/v1/...` yolunu `/api/...` olarak yeniden yazıyor, ancak rota dosyaları `app/api/v1/` altında duruyordu; yeniden yazılan yolda karşılık bulunmadığı için istekler catch-all'a düşüyordu.

Etkilenen ve düzeltilen uçlar:

Eski konum (404)	Yeni konum	Şimdi çalışan yollar
<code>app/api/v1/bootstrap</code>	<code>app/api/bootstrap</code>	<code>/api/bootstrap</code> , <code>/api/v1/bootstrap</code>
<code>app/api/v1/deeplink/resolve</code>	<code>app/api/deeplink/resolve</code>	<code>/api/deeplink/resolve</code> , <code>/api/v1/deeplink/resolve</code>
<code>app/api/v1/rtc/telemetry</code>	<code>app/api/rtc/telemetry</code>	<code>/api/rtc/telemetry</code> , <code>/api/v1/rtc/telemetry</code>

Bu bir davranış **düzeltilmesidir**: daha önce 404 dönen uçlar artık yanıt veriyor. Çalışan hiçbir uç değiştirilmedi.

7. Doğrulama

Kontrol	Sonuç
<code>tsc --noEmit</code>	✓
Üretim derlemesi	✓
<code>lib/rtc-telemetry.ts</code> çalışma zamanı testi (gerçek veritabanı)	✓ skarlama, kırpma, geçersiz girdi→null, özet, test kayıtları temizlendi
<code>POST /api/v1/rtc/telemetry</code> oturumsuz	✓ 401 UNAUTHORIZED zarfı
<code>GET /api/v1/rtc/telemetry</code>	✓ 405 (yalnızca POST tanımlı)
<code>GET /api/admin/rtc-telemetry</code> oturumsuz	✓ 401 zarfı
<code>GET /api/v1/bootstrap</code>	✓ 200
<code>GET /api/v1/deeplink/resolve</code>	✓ 200
Admin arayüzü tarayıcı testi	✗ yapılmadı
Kimlik doğrulamalı uçtan uca akış	✗ yapılmadı
Deploy	✗ yapılmadı

8. Envanter

Ölçüt	Önce	Sonra
Handler	776	778
Path	498	500
Kategori	167	170
Model	214	215
<code>guardRateLimit</code> scope	17	18

9. İstemci entegrasyonu (sonraki adım önerisi)

TRTC SDK kalite callback'lerinden (`onNetworkQuality` , `onStatistics`) 10-30 saniyede bir örnek toplanıp toplu olarak `POST /api/v1/rtc/telemetry` ucuna

gönderilmelidir. Gönderim fire-and-forget olmalı, hata kullanıcı deneyimini etkilememelidir.